Lecture 3 — DP Optimizations: Convex Hull Trick, Li Chao Tree, Divide & Conquer DP, Knuth

> National Championship Prep · Prerequisites: DP I–II (weeks 14–15), interval DP (week 27), regional DP patterns with monotone queues (regional-05), segment trees (week 23) · Estimated time: 2 × 90 min

1. Overview

Many DP recurrences have the form

```
dp[i] = min over j < i of ( dp[j] + cost(j, i) )                (1D/1D)
dp[k][i] = min over j < i of ( dp[k-1][j] + cost(j, i) )         (layered partition DP)
dp[i][j] = min over i <= m < j of ( dp[i][m] + dp[m+1][j] ) + C(i, j)   (interval DP)
```

Written directly these cost $O(n^2)$, $O(k\,n^2)$ and $O(n^3)$. At championship level n is chosen so that only a better complexity passes, and the problem is to recognize **which structural property of `cost`** makes the speed-up valid. This lecture covers the four standard tools:

Property of the transition	Tool	New complexity
`cost(j, i)` = (term in j) × (term in i) + separable parts → lines	Convex hull trick (CHT)	O(n) or O(n log n)
same, but slopes/queries in arbitrary order, or segments	Li Chao tree	O(n log C)
optimal j is monotone in i (quadrangle inequality)	Divide & conquer DP	O(k n log n)
interval DP whose split point is monotone	Knuth's optimization	$O(n^2)$

Regional-05 already covered the monotone-queue (sliding window) optimization; the World Finals track (worldfinals-09) continues with the Aliens trick (WQS binary search) and slope trick, which are often combined with the tools here.

2. Convex hull trick

2.1 From transitions to lines

Suppose `dp[i] = min_j ( dp[j] + b[j]·x[i] ) + (something depending only on i)`. For a fixed j, the expression `dp[j] + b[j]·x` is a **line** in x with slope `b[j]` and intercept `dp[j]`. So each j contributes a line, and computing dp[i] is a query: *minimum of all lines so far, evaluated at x = x[i]*.

Example (CSES 2084 — Monster Game I): you must kill monster n; killing monster i with skill factor f takes s[i]·f time and then your skill factor becomes f[i]. Then `dp[i] = min_{j<i} dp[j] + s[i]·f[j]` with `f[0] = x`, `dp[0] = 0`. Line for j: slope `f[j]`, intercept `dp[j]`; query at `s[i]`.

Recognizing the shape often needs algebra: expand squares such as `(S[i] − S[j])² = S[i]² − 2·S[i]·S[j] + S[j]²`; the cross term `−2S[j]·S[i]` gives slope `−2S[j]`, and `dp[j] + S[j]²` is the intercept.

2.2 The lower envelope

The pointwise minimum of a set of lines is a **concave, piecewise-linear** function (the lower envelope). Two facts:

1. Every line appears on the envelope in **at most one contiguous interval** of x (a line minus a concave function is convex, so the set where the line is minimal — where that difference is ≤ 0 — is an interval).
2. Walking from x = −∞ to +∞, the slopes of the envelope pieces **decrease**: for very negative x the largest slope is best, for very positive x the smallest.

So if lines arrive sorted by slope (decreasing, for minimum), the envelope can be maintained as a **stack**: when a new line c arrives, the current top b becomes useless if the intersection of c with the line a below b lies at or to the left of the intersection of a and b. Pop such lines, then push c. Every line is pushed and popped at most once: **amortized O(1) per insertion**.

2.3 Answering queries

- **Arbitrary x:** binary search for the envelope piece containing x: O(log n).
- **Monotone x** (e.g. x increasing for minimum with decreasing slopes): keep a pointer into the hull that only moves forward: amortized O(1). With a deque this also works when lines are popped from the front.

The intersection comparison multiplies two differences of coordinates. With slopes and intercepts up to $10^9 \cdot 10^9$ that overflows 64 bits, so compare in `__int128` (or `long double` if values are huge and precision is acceptable). Avoid computing intersection points with integer

division.

Maximum instead of minimum: negate all slopes and intercepts, take the minimum, negate the answer.

2.4 When the monotone CHT does not apply

If slopes arrive in arbitrary order (e.g. CSES 2085 — Monster Game II, where the monotonicity guarantees of Monster Game I are removed), the stack trick fails. Options: a dynamic "LineContainer" (lines in a `std::multiset` with intersection keys, as in KACTL), or — easier to get right — a Li Chao tree.

3. Li Chao tree

A Li Chao tree is a segment tree over the **x-domain** [LO, HI] (all possible query points, or their compressed indices). Each node stores **one line**: the line that is best at the node's midpoint among the lines that have been pushed down to that node.

Insertion of a new line `nw` at a node with stored line `cur` and midpoint `mid`:

1. If `nw` beats `cur` at `mid`, swap them. Now `cur` (stored) wins at mid.
2. The loser `nw` can still be better than `cur` on **at most one side** of mid, because two lines cross at most once. Compare them at the left endpoint: if `nw` wins at `lo`, the crossing is in the left half → recurse left; otherwise recurse right.

Query at x: walk from the root to the leaf containing x and take the minimum of the stored lines evaluated at x.

Correctness. Invariant: for every x, the optimal line at x is stored on the root-to-leaf path of x. Insertion preserves it: the line kept at the node wins at mid, and the discarded side is a half where the loser is beaten everywhere (its crossing point with the winner lies in the other half).

Complexity. O(log C) per insertion and per query, where C = HI − LO + 1. Memory: O(number of lines) nodes if created lazily, 4C for a static array. **Segments** (a line valid only for $x \in [l, r]$) are inserted into the O(log C) canonical nodes of [l, r]: $O(\log^2 C)$.

Li Chao trees are also **mergeable** (insert all lines of the smaller tree into the larger, or merge recursively), which is the core of Codeforces 932F — Escape Through Leaf, combining this lecture with small-to-large merging from Lecture 4.

4. Divide & conquer optimization

4.1 Setting

Layered partition DP: split n items into k contiguous groups minimizing the total group cost:

```
dp[c][i] = min over j < i of ( dp[c-1][j] + cost(j, i) )      (group = items j+1..i)
```

Let `opt(i)` be the (smallest) j achieving the minimum in layer c. If `opt(i) ≤ opt(i+1)` for all i, we can compute a layer in O(n log n) instead of O(n²):

```
solve(lo, hi, optLo, optHi):
    mid = (lo + hi) / 2
    scan j in [optLo, min(mid-1, optHi)] to find dp[c][mid] and opt(mid)
    solve(lo, mid-1, optLo, opt(mid))
    solve(mid+1, hi, opt(mid), optHi)
```

Complexity. Recursion depth is O(log n). On each recursion level the intervals [optLo, optHi] overlap only at endpoints, so the total scanning per level is O(n). Hence O(n log n) per layer and O(k n log n) overall — *if cost(j, i) is O(1)*. When cost needs a data structure or a "moving pointers" trick (as in Codeforces 868F, where cost = number of equal pairs in a segment), the same argument shows the pointers move O(n log n) times per layer.

4.2 When is opt monotone? The quadrangle inequality

`cost` satisfies the **quadrangle inequality (QI)** if for all a ≤ b ≤ c ≤ d:

```
cost(a, c) + cost(b, d)  ≤  cost(a, d) + cost(b, c)
```

("crossing is cheaper than nesting"). **Claim:** if cost satisfies QI, then the smallest optimal j is non-decreasing in i.

Proof. Let f_i(j) = dp[c−1][j] + cost(j, i). Take i < i′, let j = opt(i) and j′ = opt(i′) (smallest minimizers), and suppose j′ < j. Then j′ < j ≤ i < i′, and QI with (a, b, c, d) = (j′, j, i, i′) gives cost(j′, i) + cost(j, i′) ≤ cost(j′, i′) + cost(j, i). Adding dp[c−1][j′] + dp[c−1][j] to both sides: f_i(j′) + f_{i′}(j) ≤ f_{i′}(j′) + f_i(j). Since j′ is optimal for i′, f_{i′}(j′) ≤ f_{i′}(j), hence f_i(j′) ≤ f_i(j). So j′ < j is also optimal for i, contradicting that j is the smallest minimizer. ∎

Checking QI in practice. For cost(j, i) = g(S[i] − S[j]) with S non-decreasing and g convex (e.g. g(x) = x²), QI holds. Direct check for x²:

```
(Sc−Sa)² + (Sd−Sb)² − (Sd−Sa)² − (Sc−Sb)² = 2(Sd − Sc)(Sa − Sb) ≤ 0.
```

In a contest, the fastest check is often empirical: write the O(k n^2) DP, print opt(i) for random inputs, and assert monotonicity. It takes five minutes and catches wrong assumptions.

Example: CSES 2086 — Subarray Squares, split an array into k parts minimizing the sum of squared part sums. With $n \le 3000$, $k \le n$, O(k n^2) is too slow and O(k n log n) passes. (This particular cost also fits CHT per layer, giving O(k n).)

5. Knuth's optimization

5.1 Setting

Interval DP of the "optimal merge / optimal split" type:

```
dp[i][i] = 0
dp[i][j] = min over i <= m < j of ( dp[i][m] + dp[m+1][j] ) + C(i, j)
```

Examples: optimal binary search tree (Knuth's original 1971 result), merging adjacent piles where merging costs the total size, CSES 2088 — Knuth Division (splitting a block costs its sum; the reverse view of merging).

5.2 The theorem

If C satisfies the quadrangle inequality **and** is monotone under inclusion ($C(b, c) \le C(a, d)$ for $a \le b \le c \le d$), then (Yao 1980) dp also satisfies QI and the optimal split points satisfy

```
opt[i][j-1]  ≤  opt[i][j]  ≤  opt[i+1][j].
```

So when computing dp[i][j] we only scan m in [opt[i][j−1], opt[i+1][j]].

Complexity. For a fixed length L = j − i + 1, the total scan is Σ_i (opt[i+1][j] − opt[i][j−1] + 1). The sum telescopes (each opt[i+1][i+L−1] appears once with + and once with − in the next term) to at most n + (opt values range) = O(n). Summed over all L: **O(n^2)** instead of O(n^3).

C(i, j) = sum of a[i..j] with $a \ge 0$ satisfies both conditions (QI with equality, and monotone because elements are non-negative), so Knuth applies to the merging/splitting problems above.

5.3 Knuth vs D&C

Both rely on monotone optimal points. D&C needs **layers** (dp of layer c uses only layer c−1) and is O(k n log n); Knuth handles **interval** DP and gives O(n^2). A layered problem with $k \approx n$ can sometimes use a Knuth-style bound too: opt[c][i−1] ≤ opt[c][i] ≤ opt[c+1][i], giving O(n^2) total for all layers.

6. Recognizing which optimization applies

You see	Try
`dp[j] + (f(j)) · (g(i))` after expanding	CHT (monotone) or Li Chao
"split into k contiguous groups", cost convex in group size/sum	D&C DP; check QI or test opt empirically
interval DP, cost = sum over interval, merge adjacent	Knuth
"exactly k groups", k large, dp convex in k	Aliens trick (worldfinals-09) + CHT
window constraint $j \geq i - L$, cost separable	monotone queue (regional-05)

7. Reference implementations

All programs compile with `g++ -std=c++17 -O2 -Wall` and were stress-tested against $O(n^2)$/$O(n^3)$ brute force on random inputs.

7.1 Monotone CHT (minimum; slopes strictly decreasing; arbitrary queries)

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// Monotone convex hull trick for MINIMUM.
// Lines must be added in strictly DECREASING slope order; queries at any x (binary search).
struct CHT {
    vector<ll> K, M;
    // is line b useless given neighbours a (smaller index) and c (just added)?
    static bool bad(ll ka, ll ma, ll kb, ll mb, ll kc, ll mc) {
        // intersection(a,c).x <= intersection(a,b).x  <=>  (mc-ma)/(ka-kc) <= (mb-ma)/(ka-kb)
        return (__int128)(mc - ma) * (ka - kb) <= (__int128)(mb - ma) * (ka - kc);
    }
    void add(ll k, ll m) {
        while (K.size() >= 2 && bad(K[K.size() - 2], M[M.size() - 2], K.back(), M.back(), k, m)) {
            K.pop_back(); M.pop_back();
        }
        K.push_back(k); M.push_back(m);
    }
    ll query(ll x) const {
        int lo = 0, hi = (int)K.size() - 1;  // value along the hull is unimodal: find the first i
        while (lo < hi) {                    // where line i is no worse than line i+1
            int mid = (lo + hi) / 2;
            if (K[mid] * x + M[mid] <= K[mid + 1] * x + M[mid + 1]) hi = mid; else lo =
```

```
mid + 1;
        }
        return K[lo] * x + M[lo];
    }
};

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, q; cin >> n >> q;              // n lines given in strictly decreasing slope,
then q queries
    CHT h;
    for (int i = 0; i < n; i++) { ll k, m; cin >> k >> m; h.add(k, m); }
    while (q--) { ll x; cin >> x; cout << h.query(x) << "\n"; }
}
```

Equal slopes: keep only the line with the smaller intercept before calling `add` (the strict-decrease precondition).

7.2 Li Chao tree (dynamic nodes, any integer domain)

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// Li Chao tree for MINIMUM over integer x in [LO, HI].
// add: O(log C), query: O(log C). Lines may arrive in any order.
struct Line {
    ll k, m;
    ll eval(ll x) const { return k * x + m; }
};
struct LiChao {
    static const ll INF = LLONG_MAX;
    ll LO, HI;
    vector<Line> t;
    vector<int> lc, rc;  // dynamic children (0 = none)
    LiChao(ll lo, ll hi) : LO(lo), HI(hi) { t.push_back({0, INF}); lc.push_back(0);
rc.push_back(0); newNode(); }
    int newNode() { t.push_back({0, INF}); lc.push_back(0); rc.push_back(0); return
(int)t.size() - 1; }
    void add(Line nw) { add(1, LO, HI, nw); }
    void add(int v, ll lo, ll hi, Line nw) {
        ll mid = lo + (hi - lo) / 2;
        bool lef = nw.eval(lo) < t[v].eval(lo);
        bool md  = nw.eval(mid) < t[v].eval(mid);
        if (md) swap(t[v], nw);              // t[v] now wins at mid
        if (lo == hi || nw.m == INF) return;
        if (lef != md) {                     // loser can only win on the left half
            if (!lc[v]) { int c = newNode(); lc[v] = c; }
            add(lc[v], lo, mid, nw);
        } else {                             // ... or only on the right half
            if (!rc[v]) { int c = newNode(); rc[v] = c; }
            add(rc[v], mid + 1, hi, nw);
```

```
        }
    }
    ll query(ll x) const {
        int v = 1; ll lo = LO, hi = HI, res = INF;
        while (v) {
            if (t[v].m != INF) res = min(res, t[v].eval(x));
            ll mid = lo + (hi - lo) / 2;
            if (x <= mid) { v = lc[v]; hi = mid; } else { v = rc[v]; lo = mid + 1; }
        }
        return res;
    }
};

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int q; cin >> q;
    LiChao lc(-1000000, 1000000);
    while (q--) {
        int type; cin >> type;
        if (type == 1) { ll k, m; cin >> k >> m; lc.add({k, m}); }
        else { ll x; cin >> x; ll r = lc.query(x); if (r == LLONG_MAX) cout << "EMPTY\n";
else cout << r << "\n"; }
    }
}
```

The "empty" line is `{0, INF}`: slope 0 means evaluating it never overflows. The code stores `newNode()` in a local before assigning to `lc[v]`, because `newNode` may reallocate the vector that `lc[v]` refers to. Make sure |k·x + m| fits in 64 bits for every line and every x in the domain.

7.3 CSES 2084 Monster Game I with a static Li Chao tree

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// CSES 2084 "Monster Game I": dp[i] = min_{j<i} dp[j] + s[i] * f[j], f[0] = x, dp[0] =
0.
// Each j contributes the line y = f[j] * X + dp[j]; query at X = s[i].
// In Monster Game I slopes f[j] decrease and query points s[i] increase, so a monotone
// pointer walk would do; the Li Chao tree needs no monotonicity (it also solves Monster
Game II).
struct LiChao {   // min, x in [0, MAXX)
    struct Line { ll k, m; ll eval(ll x) const { return k * x + m; } };
    int N; vector<Line> t;
    LiChao(int n) : N(n), t(4 * n, Line{0, LLONG_MAX}) {}
    void add(Line nw, int v, int lo, int hi) {
        int mid = (lo + hi) / 2;
        bool lef = nw.eval(lo) < t[v].eval(lo), md = nw.eval(mid) < t[v].eval(mid);
        if (md) swap(t[v], nw);
        if (hi - lo == 1) return;
        if (lef != md) add(nw, 2 * v, lo, mid);
        else add(nw, 2 * v + 1, mid, hi);
```

```
    }
    ll query(ll x, int v, int lo, int hi) {
        ll r = t[v].eval(x);
        if (hi - lo == 1) return r;
        int mid = (lo + hi) / 2;
        return min(r, x < mid ? query(x, 2 * v, lo, mid) : query(x, 2 * v + 1, mid, hi));
    }
    void add(ll k, ll m) { add(Line{k, m}, 1, 0, N); }
    ll query(ll x) { return query(x, 1, 0, N); }
};

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n; ll x; cin >> n >> x;
    vector<ll> s(n + 1), f(n + 1);
    for (int i = 1; i <= n; i++) cin >> s[i];
    for (int i = 1; i <= n; i++) cin >> f[i];
    LiChao lc(1000001);                 // s[i] <= 1e6
    lc.add(x, 0);                       // j = 0: f[0] = x, dp[0] = 0
    ll dp = 0;
    for (int i = 1; i <= n; i++) {
        dp = lc.query(s[i]);
        lc.add(f[i], dp);
    }
    cout << dp << "\n";
}
```

This version uses half-open ranges [lo, hi) and a plain array: shorter, and the one most teams keep in the notebook when the x-domain is small.

7.4 Divide & conquer DP — CSES 2086 Subarray Squares

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// CSES 2086 "Subarray Squares": split a[1..n] into k contiguous parts,
// minimize the sum of (part sum)^2.  Divide & conquer DP: O(k n log n).
int n, k;
vector<ll> S;            // prefix sums
vector<ll> prv, cur;     // dp of previous / current layer

ll cost(int p, int i) { ll s = S[i] - S[p]; return s * s; }   // part = a[p+1..i]

// compute cur[i] for i in [lo, hi], knowing the optimal p lies in [optLo, optHi]
void solve(int lo, int hi, int optLo, int optHi) {
    if (lo > hi) return;
    int mid = (lo + hi) / 2;
    pair<ll, int> best = {LLONG_MAX, -1};
    for (int p = optLo; p <= min(mid - 1, optHi); p++)
        if (prv[p] != LLONG_MAX)
            best = min(best, {prv[p] + cost(p, mid), p});
```

```
    cur[mid] = best.first;
    int opt = best.second == -1 ? optLo : best.second;
    solve(lo, mid - 1, optLo, opt);
    solve(mid + 1, hi, opt, optHi);
}

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    cin >> n >> k;
    S.assign(n + 1, 0);
    for (int i = 1; i <= n; i++) { ll x; cin >> x; S[i] = S[i - 1] + x; }
    prv.assign(n + 1, LLONG_MAX);
    prv[0] = 0;                                  // 0 parts cover 0 elements
    cur.assign(n + 1, LLONG_MAX);
    for (int j = 1; j <= k; j++) {
        fill(cur.begin(), cur.end(), LLONG_MAX);
        solve(1, n, 0, n - 1);
        swap(prv, cur);
    }
    cout << prv[n] << "\n";
}
```

`LLONG_MAX` marks unreachable states (fewer than j items cannot form j non-empty parts); they are skipped rather than added to, which avoids overflow. Ties are broken toward the smaller p (by comparing pairs), matching the proof in §4.2.

7.5 Knuth's optimization — CSES 2088 Knuth Division

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// CSES 2088 "Knuth Division": splitting a block costs its total sum;
// dp[i][j] = min_{i<=m<j} dp[i][m] + dp[m+1][j] + sum(i..j).
// Knuth: opt[i][j-1] <= opt[i][j] <= opt[i+1][j]  =>  O(n^2) total.
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n; cin >> n;
    vector<ll> S(n + 1, 0);
    for (int i = 1; i <= n; i++) { ll x; cin >> x; S[i] = S[i - 1] + x; }
    vector<vector<ll>> dp(n + 2, vector<ll>(n + 2, 0));
    vector<vector<int>> opt(n + 2, vector<int>(n + 2, 0));
    for (int i = 1; i <= n; i++) opt[i][i] = i;
    for (int len = 2; len <= n; len++) {
        for (int i = 1; i + len - 1 <= n; i++) {
            int j = i + len - 1;
            dp[i][j] = LLONG_MAX;
            int from = opt[i][j - 1], to = min(opt[i + 1][j], j - 1);
            for (int m = from; m <= to; m++) {
                ll v = dp[i][m] + dp[m + 1][j] + (S[j] - S[i - 1]);
                if (v < dp[i][j]) { dp[i][j] = v; opt[i][j] = m; }
            }
```

```
        }
    }
    cout << dp[1][n] << "\n";
}
```

The `min(..., j - 1)` clamp handles the base case opt[i][i] = i, which would otherwise allow m = j.

8. Common mistakes

- **Wrong hull direction.** For minimum with the stack CHT, slopes must be added in decreasing order; for maximum, increasing (or negate). Mixing them silently produces wrong answers on half the tests.
- **Equal slopes** in the monotone CHT: the `bad` test divides by zero conceptually. Deduplicate (keep the better intercept).
- **Overflow in the intersection test.** Products of two 10^9–10^{12} quantities need `__int128`.
- **Li Chao domain too small** — a query outside [LO, HI] walks off the tree. Compress query points if the domain is huge and sparse.
- **Assuming monotone opt without checking.** D&C on a cost that violates QI gives wrong answers that pass small samples. Print opt from the brute force for random inputs.
- **D&C: forgetting `j < i`** (the upper bound `min(mid - 1, optHi)`), or adding to an "infinite" state.
- **Knuth: wrong base-case opt** or iterating by i instead of by length (dp[i][m] and dp[m+1][j] must already be computed).
- **Memory in Knuth:** two n × n tables of 64-bit/32-bit values for n = 5000 is ~300 MB. Check the limit; store opt as `int` (or `short` if n < 32768).
- **Using a heavy optimization when a simple one suffices.** If the queries are monotone, the pointer CHT is shorter than Li Chao.

9. Practice problems

Convex hull trick / Li Chao

1. CSES 2084 — Monster Game I
2. CSES 2085 — Monster Game II
3. SPOJ ACQUIRE — Land Acquisition (USACO 2008 March Gold; sort and remove dominated rectangles first)
4. Codeforces 319C — Kalila and Dimna in the Logging Industry
5. Codeforces 1083E — The Fair Nut and Rectangles
6. Codeforces 631E — Product Sum
7. Codeforces 932F — Escape Through Leaf (Li Chao + small-to-large; do after Lecture 4)

Divide & conquer DP 8. CSES 2086 — Subarray Squares 9. CSES 2087 — Houses and Schools 10. Codeforces 321E — Ciel and Gondolas 11. Codeforces 868F — Yet Another Minimization Problem (cost via moving pointers) 12. APIO 2014 — Split the Sequence (CHT per layer, with reconstruction)

Knuth 13. CSES 2088 — Knuth Division

10. Further reading

- cp-algorithms: Convex hull trick and Li Chao tree, Divide and conquer DP, Knuth's optimization.
- D. E. Knuth, "Optimum binary search trees", *Acta Informatica* 1 (1971).
- F. F. Yao, "Efficient dynamic programming using quadrangle inequalities", STOC 1980.
- KACTL `LineContainer` — the multiset-based dynamic CHT.
- Next in the track: Lecture 4 (tree decompositions; small-to-large merging of Li Chao trees in CF 932F). Later: worldfinals-09 (*Hard DP*: Aliens trick, slope trick, SOS DP).